



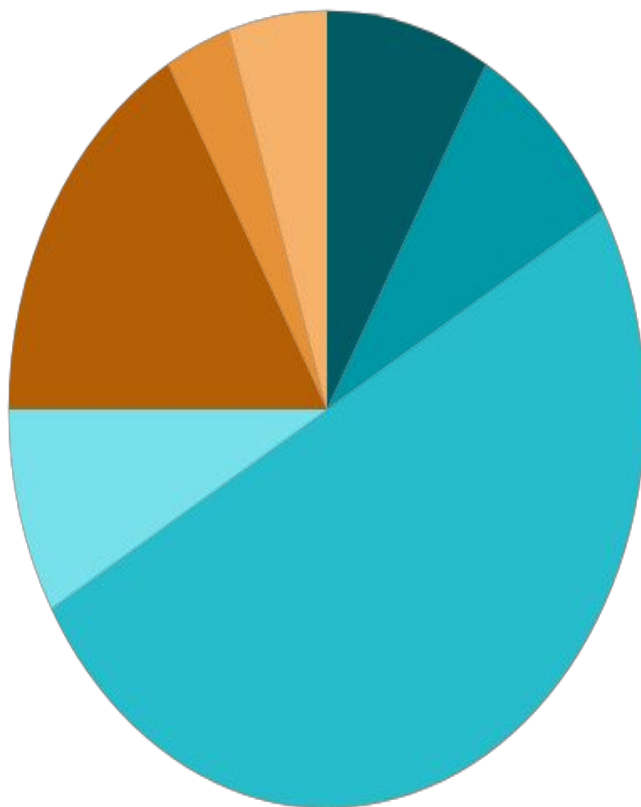
Deep Learning Foundations for Generative AI



Outline

- Introduction to Deep Learning for Generative AI Neural Network Architectures
- Review of Autoencoders
- Variational Autoencoders (VAEs)
- Loss Functions in Generative Models Summary and Discussion

Lecture Structure



**05
Minutes**

Outline & Objectives



**05
Minutes**

Recap of Previous Lecture



**30
Minutes**

**Delivery of Actual Content as per
Lesson Plan**



**05
Minutes**

Industry Relevance



**10
Minutes**

Interactive Activities



**02
Minutes**

Importance of Discipline



**03
Minutes**

Recording of Attendance

UNIT 2 • INTRODUCTION



Why This Unit Matters

- Generative AI systems (GANs, diffusion models, VAEs, LLM decoders) are all built on the neural network foundations covered in this unit.
- Before a model can generate anything, it must first learn to represent data efficiently — this is exactly what autoencoders do.
- VAEs turn that representation into a probabilistic latent space, enabling controlled generation of new samples.
- Loss functions are the mathematical compass that guides every network toward useful representations and realistic outputs.
- By the end of this unit, you will be able to design, train, and reason about the core building blocks used in almost every modern generative model.
- Each of the four building blocks in this unit maps to a specific stage of a modern generative pipeline, from raw data to sampled output.

Think of this unit as the bridge between "standard deep learning" and "generative deep learning."

UNIT 2 • INTRODUCTION

Learning Outcomes

● Architectures

Explain how feedforward, convolutional, and recurrent networks process information differently.

● Representation Learning

Describe how vanilla and sparse autoencoders compress and reconstruct data.

● Probabilistic Generation

Derive how VAEs model latent variables to generate new, realistic samples.

● Optimization

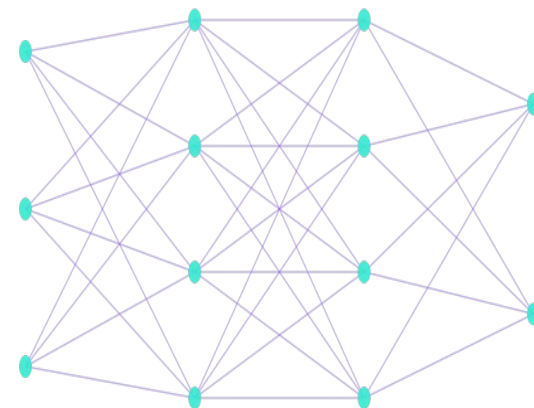
Apply cross-entropy, KL divergence, and reconstruction loss correctly during training.

01

Neural Network Architectures

Feedforward, Convolutional, and Recurrent Networks

- From the biological neuron to the artificial neuron
- Feedforward Neural Networks (FFNN) and backpropagation
- Convolutional Neural Networks (CNN) for spatial data
- Recurrent Neural Networks (RNN) for sequential data



SECTION 1 • NEURAL NETWORK ARCHITECTURES

From Biological to Artificial Neurons



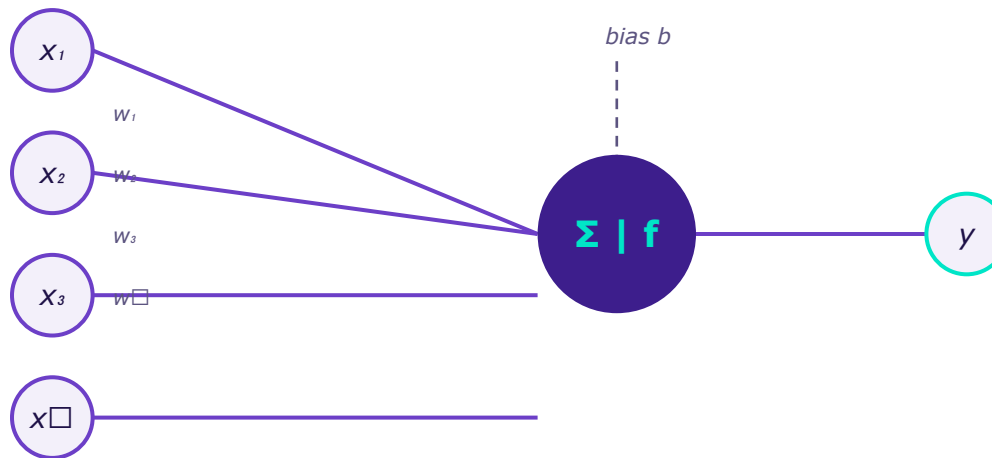
- A biological neuron receives signals through dendrites, combines them in the cell body, and fires an output signal along the axon if a threshold is crossed.
- An artificial neuron mimics this: it takes weighted inputs, sums them with a bias, and passes the result through an activation function.
- This simple unit, when connected into large networks, becomes capable of approximating extremely complex functions.
- McCulloch-Pitts (1943) and Rosenblatt's Perceptron (1958) were the earliest mathematical models of this idea.
- This biological analogy is loose but useful: modern networks do not model real neuron biology, only the abstract weighted-sum-and-threshold idea.

SECTION 1 • NEURAL NETWORK ARCHITECTURES

The Artificial Neuron (Perceptron)



$$\text{output} = f(\sum w_i x_i + b)$$



Each input x_i is multiplied by a weight w_i , summed with bias b , and passed through activation function f to produce output y .

SECTION 1 • NEURAL NETWORK ARCHITECTURES



The Perceptron: Learning Rule

- A single perceptron can only learn linearly separable functions (e.g. AND, OR) — it fails on XOR.
- Weights are updated using the rule: $w \leftarrow w + \eta(\text{target} - \text{predicted}) \cdot x$, where η is the learning rate.
- This limitation motivated stacking multiple layers of neurons — giving rise to Multi-Layer Perceptrons (MLPs), the basis of feedforward networks.
- Adding hidden layers with non-linear activations allows the network to model non-linear decision boundaries.
- Exam tip: the XOR failure of a single perceptron is the classic textbook example of a linearly inseparable problem.

Section 1 • Neural Network Architectures

7



A **perceptron** is the simplest type of artificial neural network. It takes input values, multiplies them by weights, adds a bias, and decides the output.

Mathematical Model

For inputs $x_1, x_2, \dots, x_n$,

$$y = \begin{cases} 1 & \text{if } \sum w_i x_i + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

where:

- **x** = input
- **w** = weight
- **b** = bias
- **y** = output

2. Learning Rule

The slide shows the learning rule:

$$w \leftarrow w + \eta(\text{target} - \text{predicted})x$$

This equation tells us **how the perceptron updates its weights after every training example.**

Where:

- **w** = current weight
- **η (eta)** = learning rate (small positive number like 0.1)
- **target** = actual desired output
- **predicted** = perceptron's output
- **x** = input value

The weight is changed only if there is an error.

Why Multi-Layer Perceptrons (MLPs)?

Since a single perceptron cannot solve XOR, researchers introduced **multiple layers of neurons**.

Instead of:

Input → Output

we use:

Input → Hidden Layer(s) → Output

The hidden layers learn intermediate features, allowing the network to solve complex, non-linear problems such as:

- XOR
- Image recognition
- Speech recognition
- Natural language processing

This architecture is called a **Multi-Layer Perceptron (MLP)**.



Q. Why does a single perceptron fail to solve the XOR problem?

Answer:

- XOR is **not linearly separable**.
- A single perceptron can learn only **linearly separable** patterns.
- Therefore, it cannot classify XOR correctly.
- A **Multi-Layer Perceptron (MLP)** with hidden layers and non-linear activation functions can successfully solve the XOR problem.

Key Takeaways

- A perceptron updates its weights using the learning rule: $w \leftarrow w + \eta(\text{target} - \text{predicted})x$.
- It learns from its mistakes by adjusting the weights.
- A single perceptron works for **AND** and **OR** gates but **fails for XOR** because XOR is not linearly separable.
- **Multi-Layer Perceptrons (MLPs)** overcome this limitation using hidden layers and non-linear activation functions, enabling them to solve complex real-world problems.



Feedforward Neural Networks (FFNN)

DEFINITION

A Feedforward Neural Network is a *network* where information flows in one *direction* only — from input, through hidden layers, to output — with no cycles or loops.

- Also called a Multi-Layer Perceptron (MLP) when it has one or more hidden layers.
- Each layer is typically “fully connected” (dense) to the next.
- Well suited to structured/tabular data where input order and spatial/temporal structure do not matter.
- Depth (number of hidden layers) and width (neurons per layer) are the two main levers for controlling an FFNN's representational capacity.

8

Feedforward Neural Networks (FFNN)

Definition

A **Feedforward Neural Network (FFNN)** is a neural network in which **information flows in only one direction**:

Input Layer → **Hidden Layer(s)** → **Output Layer**

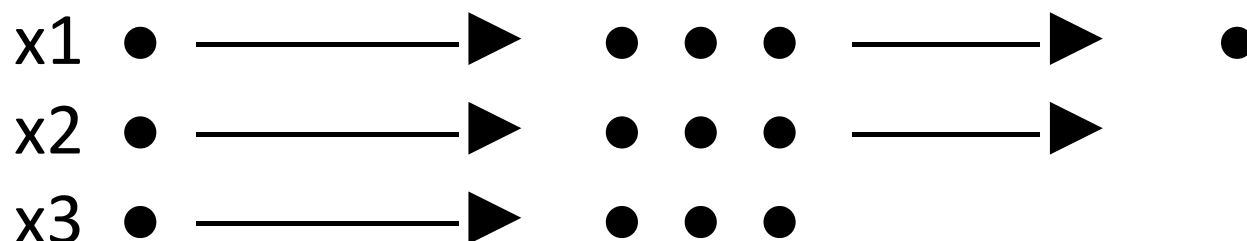
There are:

- **No loops**
- **No cycles**
- **No feedback connections**

Each neuron passes its output only to the neurons in the next layer.

Diagram

Input Layer Hidden Layer Output Layer



The data always moves **forward**, which is why it is called a **Feedforward Neural Network**.

Also Called a Multi-Layer Perceptron (MLP)

When a feedforward network has **one or more hidden layers**, it is called a **Multi-Layer Perceptron (MLP)**.

Types

Single-Layer Perceptron

Input → Output

- No hidden layer
- Can solve only linearly separable problems (e.g., AND, OR)

Multi-Layer Perceptron (MLP)

Input → Hidden Layer(s) → Output

- One or more hidden layers
- Can solve complex, non-linear problems such as XOR, image classification, and speech recognition.

Fully Connected (Dense) Layers

Each layer is typically "fully connected" (dense) to the next.

This means:

Every neuron in one layer is connected to **every neuron** in the next layer.

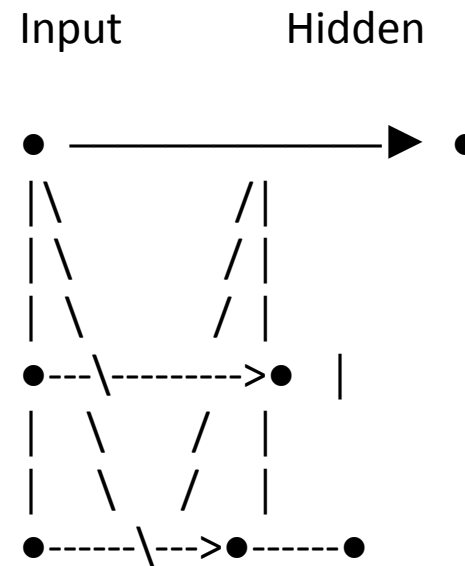
Example

Suppose

Input layer has **3 neurons**

Hidden layer has **4 neurons**

Each input neuron connects to every hidden neuron.



Advantages of FFNN

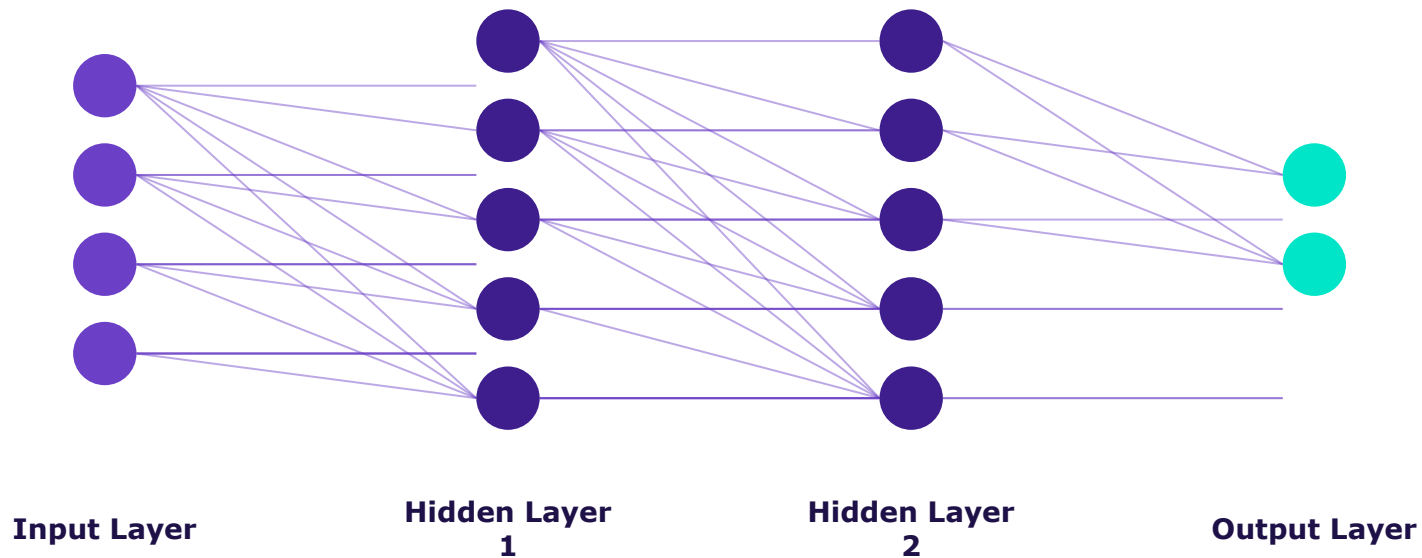
- Simple and easy to understand.
- Can approximate complex non-linear functions with hidden layers.
- Effective for structured/tabular data.
- Used for classification and regression tasks.
- Foundation for many advanced neural network architectures.

Limitations of FFNN

- Cannot remember previous inputs (no memory).
- Not suitable for sequential data like text or speech.
- Does not capture spatial information in images.
- Can become computationally expensive with many layers and neurons.

SECTION 1 • NEURAL NETWORK ARCHITECTURES

FFNN Architecture



Signals flow strictly left to right: input layer → hidden layers → output layer. No feedback connections exist.

SECTION 1 • NEURAL NETWORK ARCHITECTURES



The Forward Pass

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = f(z^{(l)})$$

- $z^{(l)}$ is the pre-activation (weighted sum) at layer l ; $a^{(l)}$ is the activation output of layer l .
- $W^{(l)}$ and $b^{(l)}$ are the learnable weight matrix and bias vector for layer l .
- f is a non-linear activation function — without it, stacking layers would collapse into a single linear transformation.
- The final layer's activation $a^{(L)}$ is the network's prediction.
- Notice the recursive structure: $a^{(l)}$ becomes the input to layer $l+1$, which is exactly why this is called a "feedforward" pass.

Step 1: Compute the weighted sum (Pre-activation)

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

where:

- $z^{(l)}$ = Pre-activation (weighted sum) of layer l
- $W^{(l)}$ = Weight matrix of layer l
- $a^{(l-1)}$ = Output (activation) from the previous layer
- $b^{(l)}$ = Bias vector

What happens?

Each neuron receives inputs from the previous layer.

Each input is multiplied by its corresponding weight.

The neuron adds all these weighted inputs together and then adds a bias.

Mathematically,

Output before activation = (Inputs $\times$ Weights) + Bias

This value is called the **pre-activation** or **weighted sum**.

Step 2: Apply the activation function

$$a^{(l)} = f(z^{(l)})$$

where:

- $a^{(l)}$ = Output (activation) of layer l
- $f(\quad)$ = Activation function

Examples of activation functions:

- ReLU
- Sigmoid
- Tanh
- Softmax (usually in the output layer)

The activation function introduces **non-linearity**, allowing the neural network to learn complex patterns.

SECTION 1 • NEURAL NETWORK ARCHITECTURES

Activation Functions (I)

Sigmoid

- $\sigma(z) = 1 / (1 + e^{-z})$
- Squashes output to (0, 1)
- Historically used in output layers for binary classification
- Suffers from vanishing gradients for large $|z|$

Tanh

- $\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$
- Squashes output to (-1, 1), zero-centered
- Stronger gradients than sigmoid near zero
- Still saturates for large $|z|$, causing vanishing gradients
- Both sigmoid and tanh are largely avoided in deep hidden layers today precisely because of vanishing gradients, though sigmoid remains common at output layers for probabilities.

SECTION 1 • NEURAL NETWORK ARCHITECTURES

Activation Functions (II)

ReLU

- $\text{ReLU}(z) = \max(0, z)$
- Cheap to compute, no saturation for $z > 0$
- Default choice for hidden layers in most modern networks
- Can suffer from “dying ReLU” when neurons output 0 permanently

Softmax (output layer)

- $\text{softmax}(z_i) = e^{z_i} / \sum_j e^{z_j}$
- Converts raw scores (logits) into a probability distribution over classes
- Used in the output layer for multi-class classification
- Pairs naturally with cross-entropy loss (covered in Section 4)
- Leaky ReLU and its variants (ELU, GELU, Swish) were developed specifically to fix the dying ReLU problem while keeping ReLU's efficiency.

SECTION 1 • NEURAL NETWORK ARCHITECTURES

Training: Loss, Gradient Descent, Backpropagation



- Step 1 — Forward pass: compute predictions and the loss (error) against the true labels.
- Step 2 — Backward pass (backpropagation): apply the chain rule to compute the gradient of the loss with respect to every weight, layer by layer, from output back to input.
- Step 3 — Update: adjust each weight in the direction that reduces the loss: $W \leftarrow W - \eta \partial L / \partial W$.
- This cycle repeats over many batches and epochs until the loss converges to a satisfactory minimum.
- This forward-backward-update cycle is often summarized as the "training loop" and underlies virtually every deep learning framework's optimizer step.

SECTION 1 • NEURAL NETWORK ARCHITECTURES

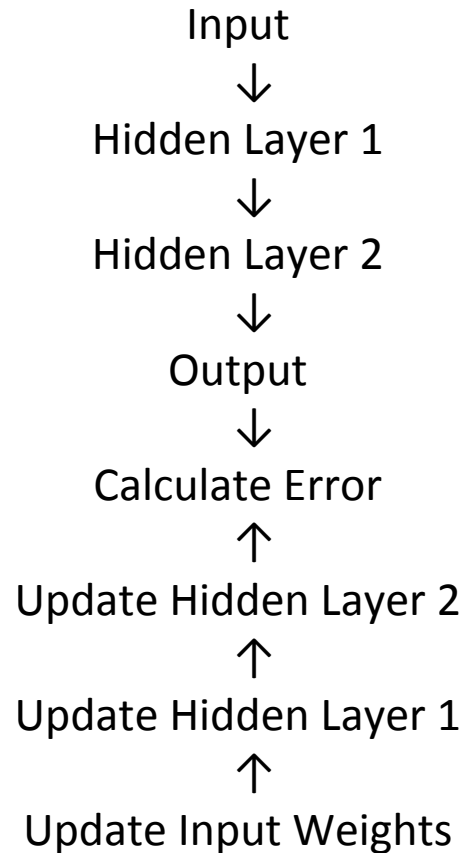
Backpropagation: The Chain Rule



$$\partial L / \partial W^{(l)} = \partial L / \partial a^{(L)} \cdot \partial a^{(L)} / \partial z^{(L)} \cdot \dots \cdot \partial z^{(l)} / \partial W^{(l)}$$

- Error signals (“deltas”) are propagated backward from the output layer to earlier layers.
- Each layer only needs the delta from the layer ahead of it and its own local derivative — making the computation efficient and modular.
- Optimizers like SGD, Momentum, RMSProp, and Adam use these gradients to update weights with varying degrees of adaptivity and stability.
- Adam is the most widely used optimizer in practice because it combines momentum (velocity) with per-parameter adaptive learning rates.

Backpropagation is an algorithm used to train neural networks by propagating the prediction error backward from the output layer to the input layer and updating the weights using the **Chain Rule** of calculus.



SECTION 1 • NEURAL NETWORK ARCHITECTURES



FFNN: Strengths and Limitations

▪ Strengths

- Simple, general-purpose, works well on structured/tabular data.
- Theoretically capable of approximating any function given enough capacity.

▪ Limitations

- Treats every input feature independently — ignores spatial structure in images.
- Has no notion of sequence or time order — unsuitable for text, speech, or time series.
- Fully connected layers scale poorly: a 256×256 image already has 65,536 inputs, exploding parameter count.

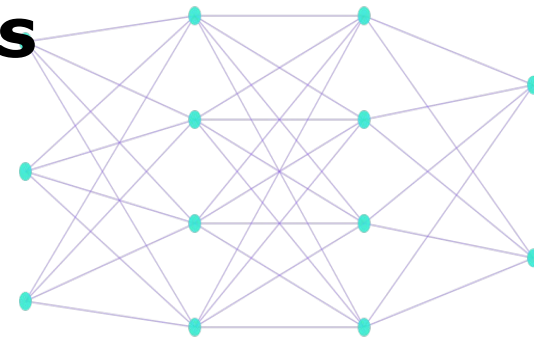
These gaps motivate the two specialized architectures we cover next: CNNs (for spatial data) and RNNs (for sequential data).

01.2

Convolutional Neural Networks

Exploiting spatial structure in images and grid-like data

- Convolution operation, filters, and feature maps
- Stride, padding, and pooling
- Typical CNN architecture and famous models



SECTION 1 • NEURAL NETWORK ARCHITECTURES



Why CNNs? Motivation

- Images have strong local spatial structure — nearby pixels are highly correlated (an edge, a texture, an eye).
- A fully connected FFNN ignores this structure and treats every pixel independently, requiring huge numbers of parameters.
- CNNs exploit two key ideas: local receptive fields (look at small patches at a time) and weight sharing (the same filter slides across the whole image).
- This makes CNNs far more parameter-efficient and gives them a useful property: translation invariance — a cat is recognized as a cat wherever it appears in the image.
- Weight sharing is what makes CNNs dramatically more parameter-efficient than a fully connected layer on the same image.

SECTION 1 • NEURAL NETWORK ARCHITECTURES

The Convolution Operation



$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m, n)$$

- I is the input image (or feature map); K is a small learnable filter/kernel (e.g. 3×3).
- The filter slides across the image; at each position it computes a weighted sum (dot product) between the filter and the underlying patch.
- The result is a new 2D grid called a feature map, highlighting where a particular pattern (edge, corner, texture) appears in the input.
- A CNN layer learns many such filters simultaneously, each detecting a different pattern.
- Stacking many convolutional layers lets early layers detect simple patterns (edges) while deeper layers combine them into complex, high-level features (faces, objects).



SECTION 1 • NEURAL NETWORK ARCHITECTURES

Stride and Padding

Stride

- Number of pixels the filter moves at each step
- Stride 1: filter moves one pixel at a time (dense overlap)
- Stride 2+: filter skips pixels, producing a smaller output map
- Larger stride → more downsampling, fewer computations

Padding

- Adding border pixels (usually zeros) around the input
- “Valid” padding: no padding, output shrinks after convolution
- “Same” padding: output size matches input size
- Prevents excessive shrinkage and preserves edge information over many layers

SECTION 1 • NEURAL NETWORK ARCHITECTURES

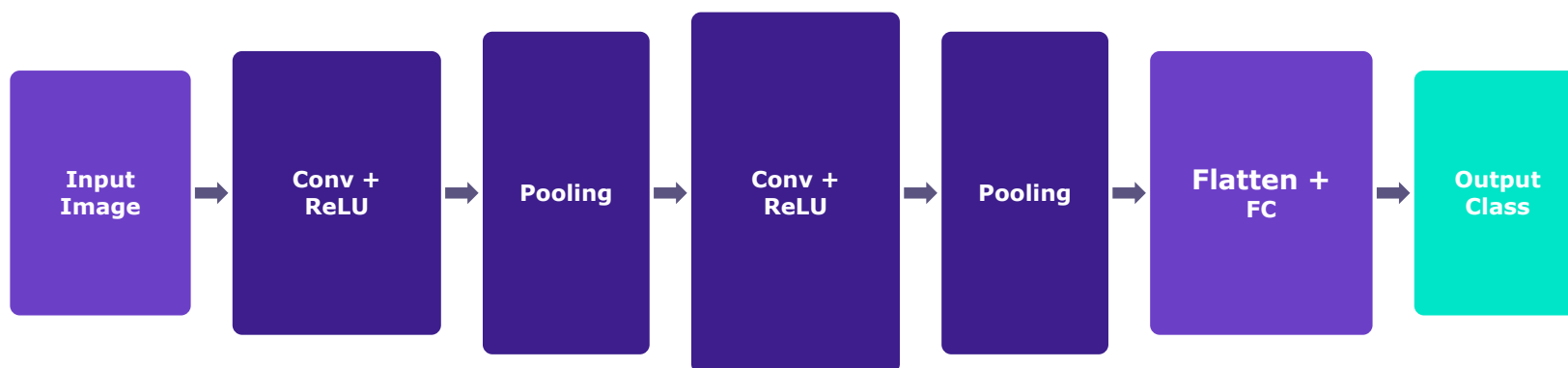


Pooling Layers

- Pooling reduces the spatial size of feature maps, cutting computation and helping control overfitting.
- **Max Pooling**
 - Takes the maximum value within each small window (e.g. 2×2) — keeps the strongest activation.
- **Average Pooling**
 - Takes the mean value within each window — gives a smoother, less aggressive summary.
- Pooling also adds a degree of translation invariance: small shifts in the input barely change the pooled output.
- Modern architectures increasingly replace pooling with strided convolutions, but pooling remains common in lighter-weight and classic CNN designs.

SECTION 1 • NEURAL NETWORK ARCHITECTURES

Typical CNN Architecture



A stack of Conv+ReLU and Pooling blocks progressively extracts higher-level features, ending in fully connected layers for classification.

SECTION 1 • NEURAL NETWORK ARCHITECTURES



Landmark CNN Architectures

- **LeNet-5 (1998)**

- One of the earliest CNNs, used for handwritten digit recognition (MNIST).

- **AlexNet (2012)**

- Deeper network with ReLU and dropout; sparked the deep learning boom by winning ImageNet by a large margin.

- **VGGNet (2014)**

- Very deep, uniform 3×3 convolution stacks; showed depth alone can improve accuracy.

- **ResNet (2015)**

- Introduced residual/skip connections, enabling networks with 100+ layers to train successfully.
- This progression — LeNet to AlexNet to VGGNet to ResNet — illustrates the broader trend of the field: networks got deeper as techniques to keep gradients stable improved.

SECTION 1 • NEURAL NETWORK ARCHITECTURES

CNN Applications



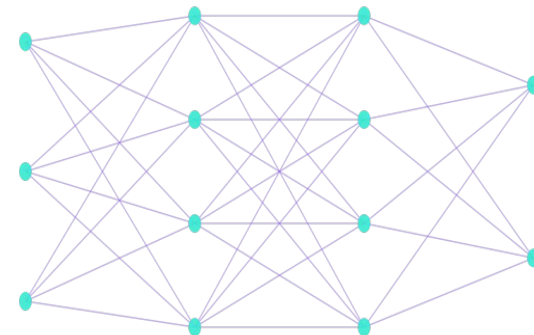
- Image classification (identifying what an image contains).
- Object detection and localization (finding where objects are).
- Semantic segmentation (labeling every pixel by class).
- Feature extraction backbones for generative models — CNN encoders/decoders appear inside GANs, VAEs, and diffusion U-Nets.
- Medical imaging, video analysis, and any data with grid-like spatial structure.
- In generative pipelines specifically, CNNs most often appear as the convolutional encoder/decoder inside image-based VAEs and as the backbone of diffusion U-Nets.

01.3

Recurrent Neural Networks

Modeling sequential and temporal data

- Hidden state and unfolding through time
- Backpropagation through time (BPTT)
- Vanishing gradients and the rise of LSTM/GRU



Unit 2 • Deep Learning Foundations for
Generative AI

25

SECTION 1 • NEURAL NETWORK ARCHITECTURES

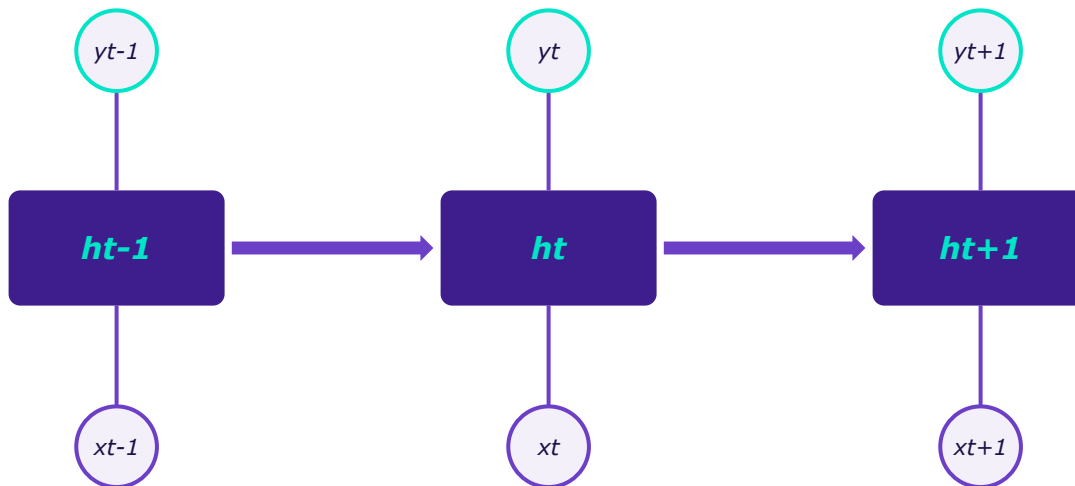
Why RNNs? Motivation



- Many real-world signals are sequential: text (word order matters), speech, audio, time-series, and even video frames.
- FFNNs and CNNs (in their basic form) process each input independently, with no memory of what came before.
- RNNs introduce a hidden state that is updated at every timestep and carried forward — giving the network a form of memory.
- The same set of weights is reused at every timestep, so the network can generalize across sequences of different lengths.
- This memory property is exactly what makes RNNs suitable for tasks where context from earlier tokens changes the meaning of later ones.

SECTION 1 • NEURAL NETWORK ARCHITECTURES

RNN Unfolded Through Time



Same weights (U , V , W) are shared and reused at every timestep

"Unrolling" the recurrent loop across timesteps reveals a deep chain of computations connected by the hidden state.

SECTION 1 • NEURAL NETWORK ARCHITECTURES

RNN Update Equations



$$h_t = f(W_{hh} h_{(t-1)} + W_{xh} x_t + b_h)$$

$$y_t = g(W_{hy} h_t + b_y)$$

- h_t is the hidden state at time t — a compressed summary of everything seen up to that point.
- W_{hh} , W_{xh} , W_{hy} are weight matrices shared across all timesteps (not one set per timestep).
- f is typically $\tanh$ or ReLU ; g depends on the task (softmax for classification, identity for regression).
- Because h_t depends on $h_{(t-1)}$, information from early in the sequence can (in principle) influence outputs much later.

SECTION 1 • NEURAL NETWORK ARCHITECTURES



Backpropagation Through Time (BPTT)

- To train an RNN, the recurrent loop is “unrolled” into a long feedforward-like chain across all timesteps.
- Gradients are computed by applying the chain rule backward through this unrolled chain — hence “through time.”
- Because the same weight matrices appear at every timestep, their gradients are summed across all timesteps before the update.
- The longer the sequence, the deeper the effective network becomes — which creates serious optimization challenges.
- BPTT is computationally expensive for long sequences, which is one practical reason truncated BPTT (limiting the unrolled window) is used in practice.

SECTION 1 • NEURAL NETWORK ARCHITECTURES



The Vanishing / Exploding Gradient Problem

KEY PROBLEM

During BPTT, gradients are repeatedly multiplied by the same recurrent weight matrix at each timestep, causing them to shrink toward zero (vanish) or grow without bound (explode) over long sequences.

- Vanishing gradients: early timesteps receive almost no learning signal — the network “forgets” long-range dependencies.
- Exploding gradients: updates become unstable, and training can diverge (often addressed with gradient clipping).
- This problem is the main reason vanilla RNNs struggle with long sequences.
- Gradient clipping is a simple, widely used fix for exploding gradients — it caps the gradient norm before the weight update is applied.

SECTION 1 • NEURAL NETWORK ARCHITECTURES

LSTM and GRU: The Fix

LSTM (Long Short-Term Memory)

- Adds a separate “cell state” that carries information with minimal transformation
- Uses input, forget, and output gates to control what information is added, kept, or discarded
- Gating mechanisms allow gradients to flow over long sequences without vanishing as quickly
- LSTM and GRU remain relevant even in the Transformer era for streaming or resource-constrained sequence tasks where full self-attention is too costly.

GRU (Gated Recurrent Unit)

- A simplified alternative with only update and reset gates
- Fewer parameters than LSTM, often trains faster
- Achieves comparable performance to LSTM on many tasks

SECTION 1 • NEURAL NETWORK ARCHITECTURES

Comparing the Three Architectures

Aspect	Feedforward (FFNN)	Convolutional (CNN)	Recurrent (RNN)
Best suited for	Tabular / structured data	Images / grid data	Sequences / time series
Key mechanism	Fully connected layers	Convolution + pooling	Recurrent hidden state
Parameter sharing	None	Filters shared spatially	Weights shared over time
Handles order/memory?	No	Local spatial context	Yes, sequential memory
Main weakness	No structure awareness	No sequence awareness	Vanishing/exploding gradients

SECTION 1 • NEURAL NETWORK ARCHITECTURES

Section 1 Recap

● **FFNN**

Fully connected layers with non-linear activations;
trained via backpropagation and gradient descent.

● **CNN**

Convolution + pooling exploit spatial locality and
weight sharing for image data.

● **RNN**

Hidden state carries memory across timesteps;
LSTM/GRU address vanishing gradients.

● **Bridge to Generative AI**

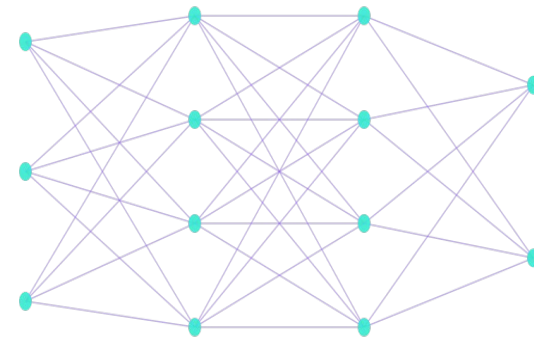
These architectures serve as the encoders/decoders
inside autoencoders, VAEs, GANs, and diffusion
models.

02

Autoencoders

Review of Vanilla and Sparse Autoencoders

- Encoder-decoder architecture and the bottleneck
- Vanilla (undercomplete) autoencoders
- Sparse autoencoders and sparsity constraints
- Applications in representation learning



SECTION 2 • AUTOENCODERS



What Is an Autoencoder?

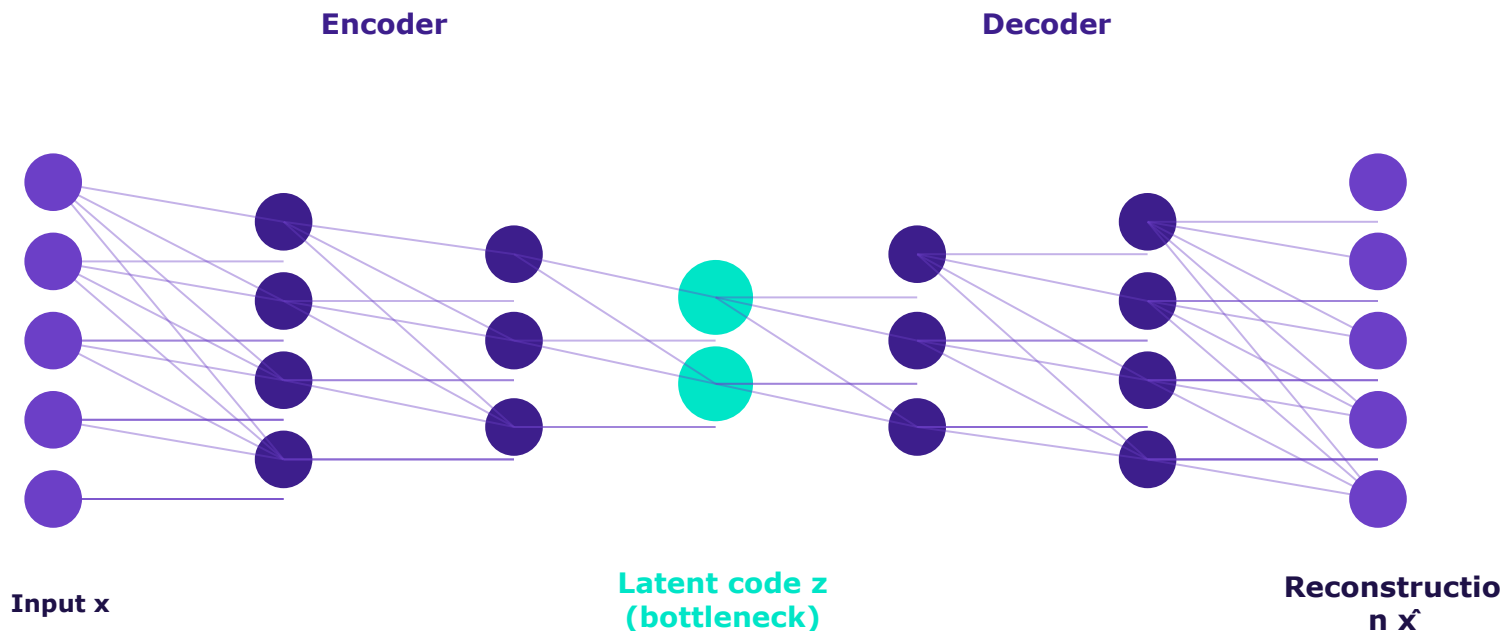
DEFINITION

An autoencoder is an unsupervised neural network trained to reconstruct its own input, forcing it to learn a compressed, meaningful internal representation of the data.

- Composed of two parts: an Encoder that compresses input into a latent code, and a Decoder that reconstructs the input from that code.
- Trained with no labels — the “target” is simply the input itself, so it is a form of self-supervised learning.
- The compressed representation (bottleneck) is useful for dimensionality reduction, denoising, and feature learning.
- Because autoencoders need no labels, they are cheap to train on large unlabeled datasets — a property that later motivates self-supervised pretraining more broadly.

SECTION 2 • AUTOENCODERS

Autoencoder Architecture



The encoder is a neural network that extracts the important features from the input and converts them into a compact latent representation, reducing the dimensionality while preserving essential information.

Section 2 • Autoencoders

36

SECTION 2 • AUTOENCODERS

Mathematical Formulation



$$z = f_{enc}(x) = \sigma(Wx + b)$$

$$\hat{x} = f_{dec}(z) = \sigma(W'z + b')$$

- f_{enc} maps the high-dimensional input x to a lower-dimensional latent code z .
- f_{dec} maps z back to the original input space, producing a reconstruction $\hat{x}$.
- The network is trained to minimize a reconstruction loss $L(x, \hat{x})$ — typically Mean Squared Error or Binary Cross-Entropy (details in Section 4).
- Ideally $\hat{x} \approx x$, meaning z retains enough information to rebuild the original input.
- In practice, encoder and decoder are usually deep multi-layer networks, not the single linear-plus-activation layer shown here; the equation captures the concept, not the full implementation.

SECTION 2 • AUTOENCODERS

Undercomplete vs. Overcomplete

Undercomplete Autoencoder

- Latent dimension is smaller than the input dimension
- Forces the network to learn only the most important/compressed features
- This is the “vanilla” autoencoder setup
- Risk: too small a bottleneck can lose important information
- Choosing undercomplete vs. overcomplete is really a choice about where the model is forced to find structure: a narrow bottleneck or an explicit sparsity constraint.

Overcomplete Autoencoder

- Latent dimension is equal to or larger than the input dimension
- Without constraints, it can simply learn the identity function (copy input to output) — not useful
- Needs an additional constraint, such as sparsity, to force useful learning
- This need motivates the Sparse Autoencoder

SECTION 2 • AUTOENCODERS



Vanilla Autoencoder: Key Characteristics

- The simplest form of autoencoder — an undercomplete encoder-decoder pair trained purely to minimize reconstruction error.
- Learns a compressed representation similar in spirit to PCA (Principal Component Analysis), but capable of capturing non-linear relationships due to activation functions.
- Typically built from fully connected (or convolutional) layers, symmetric between encoder and decoder.
- Works well for clean, well-structured data where a smooth bottleneck is sufficient.
- Unlike PCA, a vanilla autoencoder's non-linear activations let it capture curved, non-linear manifolds in the data rather than only linear subspaces.

SECTION 2 • AUTOENCODERS



Vanilla Autoencoder: Use Cases

- **Dimensionality reduction**

- Compress high-dimensional data into a compact latent space for visualization or downstream tasks.

- **Denoising**

- Train on noisy inputs but target the clean version, teaching the network to remove noise.

- **Feature learning / pretraining**

- Use the learned encoder as a feature extractor for a separate supervised task.

- **Anomaly detection**

- Data that reconstructs poorly (high reconstruction error) is flagged as anomalous.

- The same trained encoder can often be reused across several of these use cases, since the compressed representation is largely task-agnostic.

Section 2 • Autoencoders

40

SECTION 2 • AUTOENCODERS



Limitations of Vanilla Autoencoders

- The latent space has no guaranteed structure — nearby points in latent space may not decode to similar or meaningful outputs.
- Cannot be used reliably for generation: sampling a random point in latent space usually produces garbage output, since the model was never trained on “random” latent codes.
- Prone to simply memorizing training data if the bottleneck is not constrained tightly enough (especially in overcomplete settings).
- These limitations are exactly what Variational Autoencoders are designed to solve (Section 3).
- This is the central limitation the rest of the unit is built to solve: without a probabilistic latent space, sampling is unreliable.

SECTION 2 • AUTOENCODERS



Sparse Autoencoders: Motivation

KEY IDEA

*A Sparse Autoencoder allows an **overcomplete** latent representation (same or higher dimension than the input) but adds a sparsity penalty so that only a small number of neurons are active for any given input.*

- Instead of shrinking the bottleneck, we let the network have many latent neurons but force most of them to stay near zero (“inactive”) for each input.
- This encourages each neuron to specialize in detecting a specific, distinctive feature — similar to how few neurons fire for any single stimulus in biological brains.
- Sparse representations are often more interpretable and robust than dense compressed ones.
- The target sparsity p is a hyperparameter — typically a small value like 0.05, meaning each neuron should be “on” for only about 5% of inputs.

The sparse autoencoder loss should be written as:

$$L_{\text{sparse}} = L_{\text{reconstruction}}(x, \hat{x}) + \beta \sum_{j=1}^m KL(\rho \parallel \hat{\rho}_j)$$

where:

- $L_{\text{reconstruction}}(x, \hat{x})$ = reconstruction loss (MSE or Binary Cross-Entropy)
- β = sparsity penalty weight
- m = number of hidden neurons
- ρ = desired average activation (typically 0.05)
- $\hat{\rho}_j$ = actual average activation of hidden neuron j

$$KL(\rho \parallel \hat{\rho}_j) = \rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j}$$

This equation penalizes neurons whose average activation deviates from the desired sparsity level.

SECTION 2 • AUTOENCODERS

Vanilla vs. Sparse Autoencoder

Aspect	Vanilla Autoencoder	Sparse Autoencoder
Latent dimension	Smaller than input (undercomplete)	Can be equal or larger (overcomplete)
Constraint used	Bottleneck size only	Sparsity penalty (KL or L1) on activations
Neuron activity	Most neurons contribute for every input	Only a few neurons active per input
Interpretability	Lower — features are entangled	Higher — features tend to be more specialized
Risk without constraint	Information bottleneck limits capacity	Could learn identity function without sparsity

SECTION 2 • AUTOENCODERS



Other Autoencoder Variants (Brief Overview)

- **Denoising Autoencoder (DAE)**

- Trained on corrupted/noisy inputs, targets the clean version — forces robust feature learning.

- **Contractive Autoencoder (CAE)**

- Adds a penalty on the sensitivity of the latent code to small input changes, encouraging robustness.

- **Convolutional Autoencoder**

- Uses convolutional layers instead of dense layers — better suited to image data.
- All of these are variations on the same core idea: constrain the network so it must learn genuinely useful structure, not just copy the input.
- These variants show that the encoder-decoder blueprint is flexible — the reconstruction objective can be paired with almost any additional constraint to shape what the model learns.

Section 2 • Autoencoders

45



SECTION 2 • AUTOENCODERS



Autoencoders and Generative AI

- Standard autoencoders are excellent at compression and reconstruction, but poor at generating new, realistic samples from scratch.
- The core issue: their latent space has no defined structure or probability distribution to sample from meaningfully.
- Variational Autoencoders solve this by replacing the deterministic latent code with a learned probability distribution — turning a compression tool into a genuine generative model.
- This transition — from deterministic encoding to probabilistic latent variables — is the focus of the next section.
- This is the pivotal conceptual shift of the unit: adding probability turns a compression tool into a true generative model.

SECTION 2 • AUTOENCODERS

Section 2 Recap

● Core Idea

Encoder compresses input to a latent code; decoder reconstructs it. Trained by minimizing reconstruction loss.

● Vanilla AE

Undercomplete bottleneck forces compact, informative representations.

● Sparse AE

Overcomplete latent space plus a sparsity penalty (KL or L1) yields specialized, interpretable features.

● Limitation

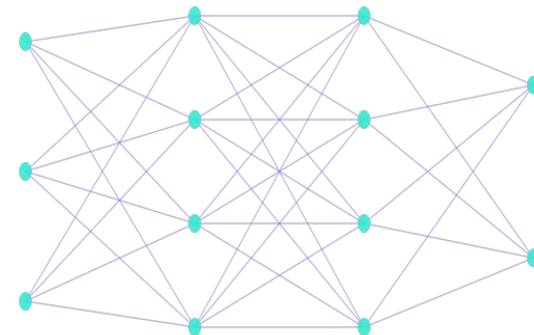
No structured latent space means standard autoencoders cannot reliably generate new samples.

03

Variational Autoencoders (VAEs)

Latent Variable Representation and Reconstruction

- From deterministic codes to probabilistic latent variables
- The reparameterization trick
- The Evidence Lower Bound (ELBO)
- Sampling, generation, and applications



SECTION 3 • VARIATIONAL AUTOENCODERS



Motivation: Why VAEs?

- A vanilla autoencoder maps each input to one fixed point in latent space — there is no notion of “nearby points should decode to similar, valid outputs.”
- If we want to generate new data, we need to sample a latent vector z and decode it — but random z values from an unconstrained latent space usually produce meaningless output.
- A Variational Autoencoder reframes the problem probabilistically: instead of learning a single latent point per input, it learns a distribution over latent variables.
- This gives the latent space a known, continuous, well-organized structure (typically Gaussian) that can be sampled from freely to generate new, realistic data.
- This reframing — point estimate to distribution — is the single most important conceptual jump in the entire unit.

SECTION 3 • VARIATIONAL AUTOENCODERS

VAEs as Latent Variable Models



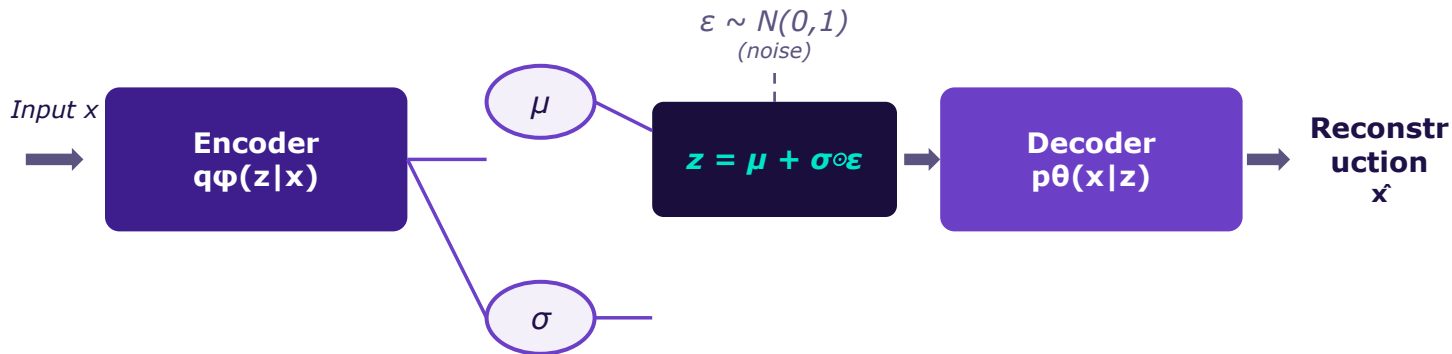
DEFINITION

A latent variable model assumes observed data x is generated from some unobserved (latent) variable z through a probabilistic process: first sample z from a prior $p(z)$, then sample x from a conditional $p(x|z)$.

- The prior $p(z)$ is usually chosen as a simple standard normal distribution $N(0, I)$.
- $p(x|z)$ is modeled by the decoder network, which defines how likely a given x is for a chosen z .
- Directly computing the true posterior $p(z|x)$ is intractable, so VAEs approximate it with a learned distribution $q\phi(z|x)$ — this is the job of the encoder.
- The intractability of $p(z|x)$ is a classic problem in Bayesian inference; VAEs sidestep it with variational inference, hence the name.

SECTION 3 • VARIATIONAL AUTOENCODERS

VAE Architecture



The reparameterization trick lets gradients flow through the sampling step during backpropagation

The encoder outputs distribution parameters (μ, σ) ; a sampled z (via reparameterization) is passed to the decoder to reconstruct x .

SECTION 3 • VARIATIONAL AUTOENCODERS



The Probabilistic Encoder

- Instead of outputting a single latent vector, the VAE encoder outputs the parameters of a distribution over z — typically the mean μ and standard deviation σ of a Gaussian.
- This approximate posterior is written $q\phi(z|x) = N(\mu(x), \sigma^2(x))$ where ϕ represents the encoder's learnable parameters.
- During training, a latent vector z is sampled from this distribution rather than being computed deterministically.
- This sampling step introduces randomness that is essential to how VAEs generate diverse outputs.
- Assuming a diagonal Gaussian (independent dimensions) keeps the KL term analytically simple, which is part of why this design choice is so common.



In a Variational Autoencoder (VAE), the encoder does not produce a single latent vector. Instead, it produces the mean (μ) and standard deviation (σ) of a probability distribution. A latent vector is then randomly sampled from this distribution, allowing the model to generate diverse and realistic new data.

SECTION 3 • VARIATIONAL AUTOENCODERS



The Reparameterization Trick

KEY TECHNIQUE

The reparameterization trick rewrites the random sampling step $z \sim N(\mu, \sigma^2)$ as $z = \mu + \sigma \odot \varepsilon$, where ε is sampled from a fixed $N(0, 1)$ distribution.

- Directly sampling z inside the computation graph is not differentiable — gradients cannot flow through a random sampling operation.
- By moving the randomness into ε (an input, not a learned parameter), μ and σ become ordinary differentiable functions of the input, and standard backpropagation works again.
- This trick is what makes end-to-end gradient-based training of VAEs possible at all.
- Without this trick, VAEs would have to rely on much noisier gradient estimators (like the score-function/REINFORCE estimator), which converge far more slowly.

Section 3 • Variational Autoencoders

53

SECTION 3 • VARIATIONAL AUTOENCODERS

Reparameterization: The Math



$$z = \mu(x) + \sigma(x) \odot \epsilon, \quad \epsilon \sim N(0, I)$$

- $\odot$ denotes elementwise multiplication.
- $\mu(x)$ and $\sigma(x)$ are computed deterministically by the encoder network and are fully differentiable.
- ϵ is pure noise, sampled once per forward pass, and treated as a constant during backpropagation.
- Gradients now flow cleanly through μ and σ , letting the encoder be trained jointly with the decoder via standard gradient descent.
- Because ϵ carries all the randomness and is never a function of trainable parameters, backpropagation can treat $\mu(x)$ and $\sigma(x)$ exactly like any other differentiable layer output.

SECTION 3 • VARIATIONAL AUTOENCODERS



The Decoder: Generating Reconstructions

- The decoder network defines $p_{\theta}(x|z)$ — the probability of the data given a latent vector z , parameterized by decoder weights θ .
- Given a sampled z , the decoder produces a reconstruction x' (for images, often the parameters of a per-pixel Bernoulli or Gaussian distribution).
- During training, z comes from encoding a real input x . During generation, z can instead be sampled directly from the prior $p(z) = N(0, I)$.
- Because the decoder was trained to map “typical” latent vectors to realistic outputs, sampling fresh z values from the prior now produces new, plausible data.
- This train-with-real- z , generate-with-prior- z asymmetry is the key trick that lets a single decoder serve both a reconstruction role and a generation role.



SECTION 3 • VARIATIONAL AUTOENCODERS



Structuring the Latent Space

- The KL divergence term (introduced properly in Section 4) pulls every encoded distribution $q\phi(z|x)$ toward the standard normal prior $N(0, I)$.
- This regularization keeps the latent space continuous and densely packed — there are no “gaps” of meaningless latent codes between real data points.
- As a result, moving smoothly through latent space (interpolation) produces smoothly changing, semantically meaningful outputs.
- This structured, continuous latent space is the single biggest advantage VAEs have over vanilla autoencoders.
- Without this KL regularization, a VAE would behave almost exactly like a vanilla autoencoder with added noise — structure comes entirely from this term.

SECTION 3 • VARIATIONAL AUTOENCODERS

Latent Space Interpolation



- Given two latent vectors z_1 and z_2 (encodings of two real inputs), we can compute intermediate points $z_t = (1-t) \cdot z_1 + t \cdot z_2$ for t between 0 and 1.
- Decoding each z_t produces a smooth sequence of outputs that gradually morph from the first input to the second.
- This works reliably in VAEs because of the smooth, continuous latent space enforced during training — it typically fails in vanilla autoencoders.
- Interpolation is a common way to visually verify that a VAE has learned a meaningful latent representation.
- Latent space arithmetic (e.g. adding a "smile vector") is a related and popular demonstration of the same underlying structured-space property.

SECTION 3 • VARIATIONAL AUTOENCODERS

The Evidence Lower Bound (ELBO)



KEY CONCEPT

Since directly maximizing the true data likelihood $p(x)$ is intractable, VAEs instead maximize a tractable lower bound on it — the Evidence Lower Bound (ELBO).

- $ELBO(x) = E_{q\phi(z|x)}[\log p_{\theta}(x|z)] - KL(q\phi(z|x) || p(z))$
- The first term rewards accurate reconstruction; the second term penalizes the encoder's distribution for straying too far from the prior.
- Maximizing the ELBO is mathematically equivalent to minimizing reconstruction loss plus KL divergence — this becomes the VAE's training objective (detailed in Section 4).

SECTION 3 • VARIATIONAL AUTOENCODERS



Training a VAE: Step by Step

- **1. Encode**
 - Pass input x through the encoder to obtain $\mu(x)$ and $\sigma(x)$.
- **2. Sample**
 - Draw $\varepsilon \sim N(0, I)$ and compute $z = \mu + \sigma \odot \varepsilon$ (reparameterization trick).
- **3. Decode**
 - Pass z through the decoder to obtain a reconstruction $\hat{x}$.
- **4. Compute Loss and Backpropagate**
 - Combine reconstruction loss and KL divergence into a single loss and update all weights via gradient descent.



SECTION 3 • VARIATIONAL AUTOENCODERS



Generating New Samples

- Once trained, the encoder can be discarded entirely for generation — only the decoder and the prior are needed.
- Step 1: Sample a fresh latent vector z directly from the prior, $z \sim N(0, I)$.
- Step 2: Pass z through the trained decoder to obtain a brand-new synthetic sample $\hat{x}$.
- Because the training process shaped the latent space to match this exact prior, decoded samples look statistically similar to the real training data.
- This is why VAEs are called generative: after training, new data can be produced without ever looking at a real input again.

SECTION 3 • VARIATIONAL AUTOENCODERS

Vanilla Autoencoder vs. VAE

Aspect	Vanilla Autoencoder	Variational Autoencoder
Latent representation	Single deterministic vector z	Distribution $q\phi(z x)$, typically Gaussian
Training objective	Reconstruction loss only	Reconstruction loss + KL divergence (ELBO)
Latent space structure	Unstructured, may have gaps	Continuous, densely packed, matches prior
Generation capability	Poor — random z rarely decodes well	Strong — designed for sampling from prior
Key mechanism	Direct encoding	Reparameterization trick for differentiable sampling

SECTION 3 • VARIATIONAL AUTOENCODERS



Applications of VAEs

- Image generation — producing new, realistic-looking images similar to a training set (faces, digits, objects).
- Anomaly and outlier detection — inputs with unusually high reconstruction error or low likelihood under the model are flagged as anomalies.
- Data imputation and denoising — filling in missing values or removing noise using the learned generative model.
- Representation learning — the structured latent space is useful as compact features for downstream tasks.
- Drug discovery and molecule generation — sampling novel molecular structures from a learned latent space.
- In each of these applications, the same core mechanism is reused — only the input data type and decoder architecture change.

SECTION 3 • VARIATIONAL AUTOENCODERS



Limitations of VAEs

- Generated samples (especially images) often look noticeably blurrier than those from GANs or diffusion models — a known side effect of the reconstruction loss and Gaussian assumptions.
- Balancing the reconstruction term and the KL term is delicate: too much KL weight collapses the latent space (posterior collapse), too little weakens the regularization benefit.
- The Gaussian prior assumption may not perfectly match the true structure of complex real-world data.
- These trade-offs have motivated further research: Beta-VAE (tunable KL weighting), Conditional VAE (class-conditioned generation), and hybrid VAE-GAN architectures.
- Understanding these limitations is exactly what motivates the next generation of generative models covered in the course: GANs and diffusion models.

SECTION 3 • VARIATIONAL AUTOENCODERS

Section 3 Recap

● Core Idea

Learn a distribution $q\phi(z|x)$ over latent variables instead of a single fixed code.

● Reparameterization

$z = \mu + \sigma \odot \epsilon$ makes the sampling step differentiable, enabling backpropagation.

● ELBO Objective

Maximize reconstruction likelihood while regularizing the latent distribution toward the prior via KL divergence.

● Generative Power

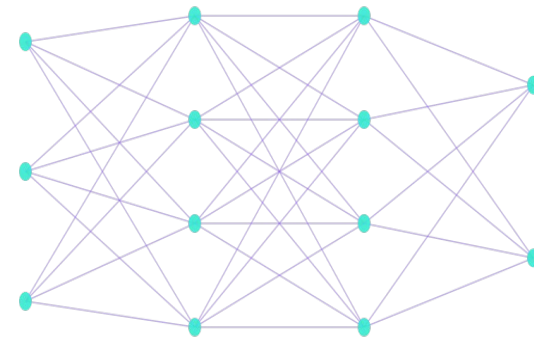
A structured, continuous latent space allows sampling entirely new, realistic data from the prior.

04

Loss Functions

Cross-Entropy, KL Divergence, and Reconstruction Loss

- Cross-entropy loss for classification and binary outputs
- KL divergence for comparing probability distributions
- Reconstruction loss for autoencoders and VAEs
- Combining losses into the VAE training objective



SECTION 4 • LOSS FUNCTIONS



Why Loss Functions Matter

- A loss function quantifies how far a model's output is from the desired result — it is the single signal that gradient descent optimizes.
- Different tasks require fundamentally different notions of “error”: classification error is not measured the same way as pixel reconstruction error.
- In generative models specifically, the loss function must balance multiple competing goals — e.g. accurate reconstruction versus a well-structured latent space.
- A poor choice of loss function can cause a network to converge to a solution that satisfies the mathematical objective but is practically useless (e.g. blurry images, mode collapse).
- Choosing the loss function is not a minor implementation detail — it directly defines what "good" means for the model during training.

SECTION 4 • LOSS FUNCTIONS



Categories of Loss Functions

- **Classification losses**

- Measure how well predicted class probabilities match true labels — e.g. Cross-Entropy Loss.

- **Distributional / divergence losses**

- Measure how different two probability distributions are — e.g. KL Divergence.

- **Regression / reconstruction losses**

- Measure numeric distance between predicted and true values — e.g. MSE, used as reconstruction loss.
- Generative models like VAEs typically combine losses from more than one of these categories into a single training objective.
- Recognizing which category, a loss belongs to helps predict its behavior: divergence losses regularize distributions, while regression losses reward pointwise accuracy.

Section 4 • Loss Functions

67

SECTION 4 • LOSS FUNCTIONS



Cross-Entropy Loss: Intuition

DEFINITION

Cross-entropy measures the difference between two probability distributions — the true distribution of labels and the model's predicted distribution — by quantifying how many extra "bits" of surprise the prediction introduces.

- It heavily penalizes confident wrong predictions (predicting near-zero probability for the correct class) and rewards confident correct predictions.
- It is the standard loss for classification tasks and any task where the output represents a probability.
- It pairs naturally with a softmax (multi-class) or sigmoid (binary) output layer.
- Cross-entropy has a strong information-theoretic interpretation: it equals the entropy of the true distribution plus the KL divergence between true and predicted distributions.

SECTION 4 • LOSS FUNCTIONS



Cross-Entropy: The Math

Binary: $L = -[y \log(p) + (1-y) \log(1-p)]$

Categorical: $L = - \sum_c y_c \log(p_c)$

- y is the true label (0 or 1 for binary; a one-hot vector for categorical), and p is the model's predicted probability.
- For binary cross-entropy, when $y = 1$, the loss reduces to $-\log(p)$ — which grows very large as $p \rightarrow 0$ (confidently wrong).
- For categorical cross-entropy, only the term for the true class c contributes (since $y_c = 0$ for all other classes in a one-hot vector).
- Minimizing cross-entropy is mathematically equivalent to maximizing the log-likelihood of the correct labels under the model.
- This equivalence to maximum likelihood estimation is why cross-entropy is the default choice whenever a model outputs a probability distribution.

SECTION 4 • LOSS FUNCTIONS



Cross-Entropy in Autoencoders

- When input data is binary or normalized to $[0, 1]$ (e.g. black-and-white pixel intensities), Binary Cross-Entropy (BCE) is a natural choice for the reconstruction loss.
- Each output pixel is treated as the parameter of a Bernoulli distribution, and BCE measures how well the reconstructed probability matches the true pixel value.
- BCE reconstruction loss is standard in many VAE implementations for image data (e.g. MNIST digits).
- For continuous, real-valued data (not bounded in $[0,1]$), Mean Squared Error is typically preferred instead (covered later in this section).
- A common implementation detail: BCE is typically summed or averaged over all pixels/features to get a single scalar loss per sample.

SECTION 4 • LOSS FUNCTIONS

KL Divergence: Intuition



DEFINITION

Kullback-Leibler (KL) Divergence measures how one probability distribution Q differs from a reference distribution P — essentially, the extra “surprise” or inefficiency incurred by assuming Q when the true distribution is P .

- $KL(P || Q) = 0$ only when P and Q are exactly identical distributions.
- The larger the divergence, the more the two distributions differ in shape, location, or spread.
- In deep learning, KL divergence is used whenever we need to compare a learned/approximate distribution against a target or prior distribution.
- KL divergence is not symmetric, so $KL(P||Q)$ answers a different question than $KL(Q||P)$ — the direction chosen changes what kind of approximation error is penalized.

SECTION 4 • LOSS FUNCTIONS

KL Divergence: The Math



$$KL(P \parallel Q) = \sum_x P(x) \log(P(x) / Q(x))$$

- Sums (or integrates, for continuous variables) the log-ratio of the two distributions, weighted by how likely each event is under P.
- KL divergence is always ≥ 0 (Gibbs' inequality), and equals 0 only when $P = Q$ everywhere.
- It is asymmetric: $KL(P \parallel Q) \neq KL(Q \parallel P)$ in general — it is not a true distance metric.
- This asymmetry matters in practice: $KL(q_\phi(z|x) \parallel p(z))$, as used in VAEs, penalizes the encoder differently than the reverse direction would.

SECTION 4 • LOSS FUNCTIONS



Key Properties of KL Divergence

- **Non-negativity**

- $KL(P||Q) \geq 0$ always; equality holds only when the two distributions are identical.

- **Asymmetry**

- $KL(P||Q)$ is generally not equal to $KL(Q||P)$ — direction matters, unlike a distance metric.

- **Sensitivity to support**

- If $Q(x) = 0$ where $P(x) > 0$, the divergence can become infinite — Q must “cover” the support of P .
- These properties explain why KL divergence, while extremely useful, must be applied thoughtfully rather than treated like an ordinary distance.

SECTION 4 • LOSS FUNCTIONS



KL Divergence in VAEs

- In a VAE, KL divergence regularizes the encoder's output distribution $q\phi(z|x)$ to stay close to the chosen prior $p(z)$, usually a standard normal $N(0, I)$.
- Without this term, the encoder could place each input's latent distribution anywhere, with no shared structure — collapsing back to something like a vanilla autoencoder.
- With this term, all encoded distributions are pulled toward a common, well-behaved region of latent space, making it safe to sample from the prior during generation.
- This is the mathematical mechanism underlying everything discussed about "structured latent space" in Section 3.
- This is the exact mechanism referenced back in Section 3 whenever "structured latent space" was mentioned without derivation.

SECTION 4 • LOSS FUNCTIONS



Closed-Form KL for Two Gaussians

$$KL(N(\mu, \sigma^2) || N(0, 1)) = \frac{1}{2} \sum_i (\sigma_i^2 + \mu_i^2 - 1 - \log(\sigma_i^2))$$

- Because both the approximate posterior $q\phi(z|x)$ and the prior $p(z)$ are Gaussian, this integral has a simple, exact closed-form solution — no numerical approximation needed.
- This is a major practical reason VAEs use Gaussian distributions: the KL term becomes a cheap, differentiable formula computed directly from μ and σ .
- The term is minimized when $\mu \rightarrow 0$ and $\sigma \rightarrow 1$ for every dimension — exactly matching the standard normal prior.

SECTION 4 • LOSS FUNCTIONS

Reconstruction Loss: Intuition



DEFINITION

Reconstruction loss measures how closely a model's reconstructed output $\hat{x}$ matches the original input x — it is the core signal that teaches an autoencoder or VAE decoder to preserve meaningful information.

- It is computed between the original data and the decoder's output at the end of the forward pass.
- The specific formula chosen (MSE vs. BCE, etc.) depends on the nature of the data being reconstructed.
- In a VAE, reconstruction loss forms one half of the total training objective, paired with the KL divergence term.
- Reconstruction loss alone, without any regularization, is exactly what a vanilla autoencoder optimizes — VAEs add the KL term on top of it.

SECTION 4 • LOSS FUNCTIONS

Choosing a Reconstruction Loss

Mean Squared Error (MSE)

- $MSE = (1/n) \sum (x_i - \hat{x}_i)^2$
- Used for continuous, real-valued data
- Assumes Gaussian noise/output distribution
- Penalizes large errors more heavily (squared term)

Binary Cross-Entropy (BCE)

- $BCE = -[x \log(\hat{x}) + (1-x) \log(1-\hat{x})]$
- Used for data normalized to $[0, 1]$ (e.g. pixel intensities)
- Assumes a Bernoulli output distribution per pixel/feature
- Standard choice in many VAE image implementations

SECTION 4 • LOSS FUNCTIONS

The Full VAE Loss (Negative ELBO)



Reconstruction Loss

How well $\hat{x}$ matches x
(BCE OR MSE)

+

KL Divergence

How close $q\phi(z|x)$ is to
prior $N(0, I)$

=

VAE Loss (–ELBO)

Minimized jointly during
training via backprop

The two terms are added together and minimized jointly — accurate reconstruction and a well-regularized latent space are learned simultaneously.

SECTION 4 • LOSS FUNCTIONS

Putting It All Together



$$L_VAE = L_reconstruction(x, \hat{x}) + KL(q\phi(z|x) || p(z))$$

- This is the practical loss function minimized during VAE training — the negative of the ELBO introduced in Section 3.
- The reconstruction term (BCE or MSE) pulls the decoder toward faithfully rebuilding the input.
- The KL term pulls the encoder's distribution toward the prior, keeping the latent space smooth and sample-ready.
- Both terms are computed on the same forward pass and backpropagated together — a single optimizer step updates encoder and decoder jointly.
- This single equation is the practical takeaway of the entire unit: everything from Sections 1-3 exists to make this loss computable and differentiable.

SECTION 4 • LOSS FUNCTIONS



Balancing the Two Loss Terms

- In practice, the reconstruction and KL terms can pull in competing directions, so their relative weight matters a great deal.
- Beta-VAE introduces a weighting hyperparameter β : $L = L_{\text{reconstruction}} + \beta \cdot \text{KL}(\dots)$ — increasing β encourages a more structured, disentangled latent space at some cost to reconstruction sharpness.
- Too high a KL weight can cause “posterior collapse,” where the encoder ignores the input entirely and the latent distribution matches the prior trivially.
- Too low a KL weight weakens the latent space regularization, degrading generation quality even though reconstructions look sharp.
- Practitioners often monitor both loss terms separately during training, not just their sum, to diagnose this balance.

SECTION 4 • LOSS FUNCTIONS

Loss Functions at a Glance

Loss Function	Measures	Typical Use	Key Property
Cross-Entropy	Difference between predicted and true class probabilities	Classification, binary/BCE reconstruction	Penalizes confident wrong predictions heavily
KL Divergence	Difference between two full probability distributions	Regularizing VAE latent space to prior	Non-negative, asymmetric
Reconstruction Loss (MSE/BCE)	Distance between input and reconstructed output	Autoencoders, VAE decoder training	Directly reflects output fidelity

81

SECTION 4 • LOSS FUNCTIONS

Section 4 Recap

● Cross-Entropy

Measures classification/probability error; heavily penalizes confident mistakes.

● KL Divergence

Measures how one distribution differs from another; regularizes VAE latent space toward the prior.

● Reconstruction Loss

MSE or BCE measures how faithfully a decoder rebuilds the original input.

● VAE Total Loss

Reconstruction loss + KL divergence, jointly minimized as the negative ELBO.

UNIT 2 • WRAP-UP

Unit 2: The Complete Picture

Building Block	Core Question It Answers	Feeds Into
Neural Network Architectures	How can a network process this type of data (tabular, image, sequence)?	Encoder/decoder design in AEs and VAEs
Autoencoders	How can a network learn a compact, meaningful representation?	The encoder-decoder blueprint for VAEs
Variational Autoencoders	How can that representation become a true generative model?	Modern generative AI systems
Loss Functions	How do we mathematically guide learning toward the right answer?	Training objective for every model above

UNIT 2 • WRAP-UP



Looking Ahead: Beyond VAEs

- Generative Adversarial Networks (GANs) — pit a generator against a discriminator in a competitive game, often producing sharper images than VAEs.
- Diffusion Models — learn to reverse a gradual noising process, currently the dominant approach for state-of-the-art image and video generation.
- Autoregressive / Transformer-based generative models — generate sequences (text, audio) one token at a time, powering modern large language models.
- All of these later architectures still rely on the encoder/decoder building blocks, loss function principles, and latent variable thinking introduced in this unit.
- GANs, diffusion models, and Transformer-based generators are covered in depth in the units that follow this one.

UNIT 2 • WRAP-UP



Suggested Readings & References

- Goodfellow, I., Bengio, Y., & Courville, A. — “Deep Learning” (MIT Press) — Chapters on feedforward networks, CNNs, RNNs, and autoencoders.
- Kingma, D. P., & Welling, M. (2013) — “Auto-Encoding Variational Bayes” — the original VAE paper introducing the reparameterization trick and ELBO.
- Bishop, C. M. — “Pattern Recognition and Machine Learning” — background on KL divergence and probabilistic modeling.
- CS231n (Stanford) and CS229 course notes — accessible treatments of CNNs, RNNs, and loss functions with worked examples.
- Reading the original VAE paper (Kingma & Welling, 2013) alongside these slides is strongly recommended for exam preparation on the reparameterization trick.

In every lecture, Faculty may talk on any of the following:

- **DCS & GSFCU Portal:** Remind students to use DCS, talk about it's features and instruct all to visit GSFCU website
- **Discipline, Values and Ethics.**
- **GSFC University – My University:** Emphasize the importance of taking ownership of the University.
- **Respect & Courtesy:** Remind students to be respectful to peers, teachers, and staff. Encourage polite language and behavior.
- **Punctuality:** Emphasize the importance of arriving on time to class and completing assignments by deadlines.
- **Classroom Rules:** Reinforce basic classroom rules, such as keeping mobile phones on silent and maintaining a clean environment.
- **Safety:** Remind students about safety protocols, whether it's physical safety in the classroom or digital safety when using online resources.
- **Positive Attitude:** Encourage a positive mindset, cooperation and supporting one another.

GSFCU & Values

Respect & Courtesy

Font

Type: Verdana
Size: 20 / 18 / 16
Colour: Black



Dress
Appropriately



Respect
Personal Space



Be Mindful
of Your Words



Respect
Coworkers' Time



Keep Your
Workplace Clean



Socialise with
Your Peers



Use Your
Mobile Wisely



Take
Responsibility

*The
End*